

GITHUB

GITHUB NEDİR?

- GitHub, yazılım projelerimizi internet üzerinde saklamamıza, yönetmemize ve başkalarıyla ortaklaşa geliştirmemize olanak tanıyan Git tabanlı bir bulut platformudur.

NEDEN GITHUB KULLANIYORUZ?

- **Versiyon Kontrolü:** Kodunuzda yaptığınız her değişikliği (Commit) kaydeder. Bir hata yaptığınızda projenin 2 gün önceki çalışan haline saniyeler içinde geri dönebilirsiniz.
- **İşbirliği:** Bir projede 10 kişi aynı anda çalışabiliriz. Herkes kendi bölümünü (Branch) geliştirir ve en sonunda ana projede (Main) birleştiririz.
- **Şeffaflık ve Kod İncelemesi:** Kimin hangi satırı neden değiştirdiğini görebilir, ekip arkadaşlarımızın kodlarını inceleyerek birbirimize geri bildirim verebiliriz.
- **Uzaktan Erişim:** Bilgisayarınıza bir şey olsa bile, kodlarınız GitHub üzerinde (Remote Repository) güvendedir ve dünyanın her yerinden erişilebilir.



ORGANİZASYON YÖNETİMİ

1. Organizasyon Kurulumu

2. Takım (Team) Mimarisi

3. Üye Yönetimi

People

Organization permissions ?

Members 6

Security Managers

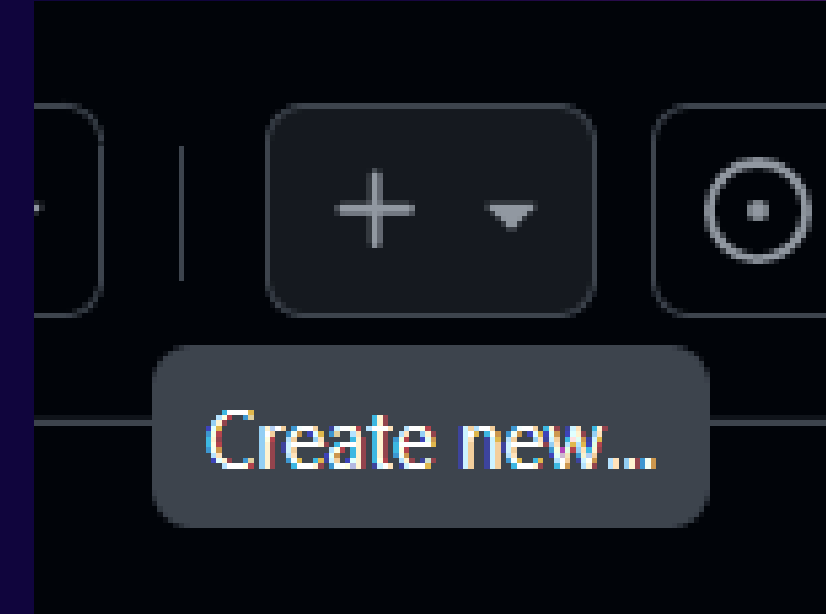
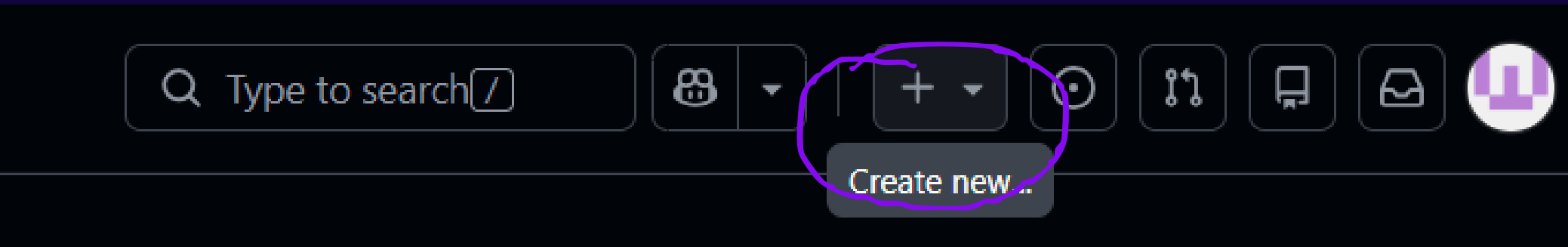
6 people in the birbucukformula-Software organization

Membership ▼

	abdulmohammed4338-prog	 Private	Member	1 team
	KruesnaPy KruesnaGod	 Private	Member	1 team
	NecdetOzdemir	Public	Owner	4 teams
	Murat Can Ventux78	 Private	Member	2 teams
	yyaseminkuru-cpu	 Private ▼	Member	1 team
	Zuleyhanurgunes	 Private	Member	2 teams

ORGANİZASYON (ORGANIZATION) NEDİR?

GitHub Organizasyonu, projelerin ve insanların toplandığı kurumsal bir çatıdır. Bu, kişisel hesapların üzerinde, kurumsal bir çatı oluşturur.



NASIL KURULUR?

1. GitHub ana sayfasında sağ üstteki + ikonuna tıkla.
2. "New organization" seçeneğini seç.
3. Bir plan (Free yeterlidir) ve isim belirle.

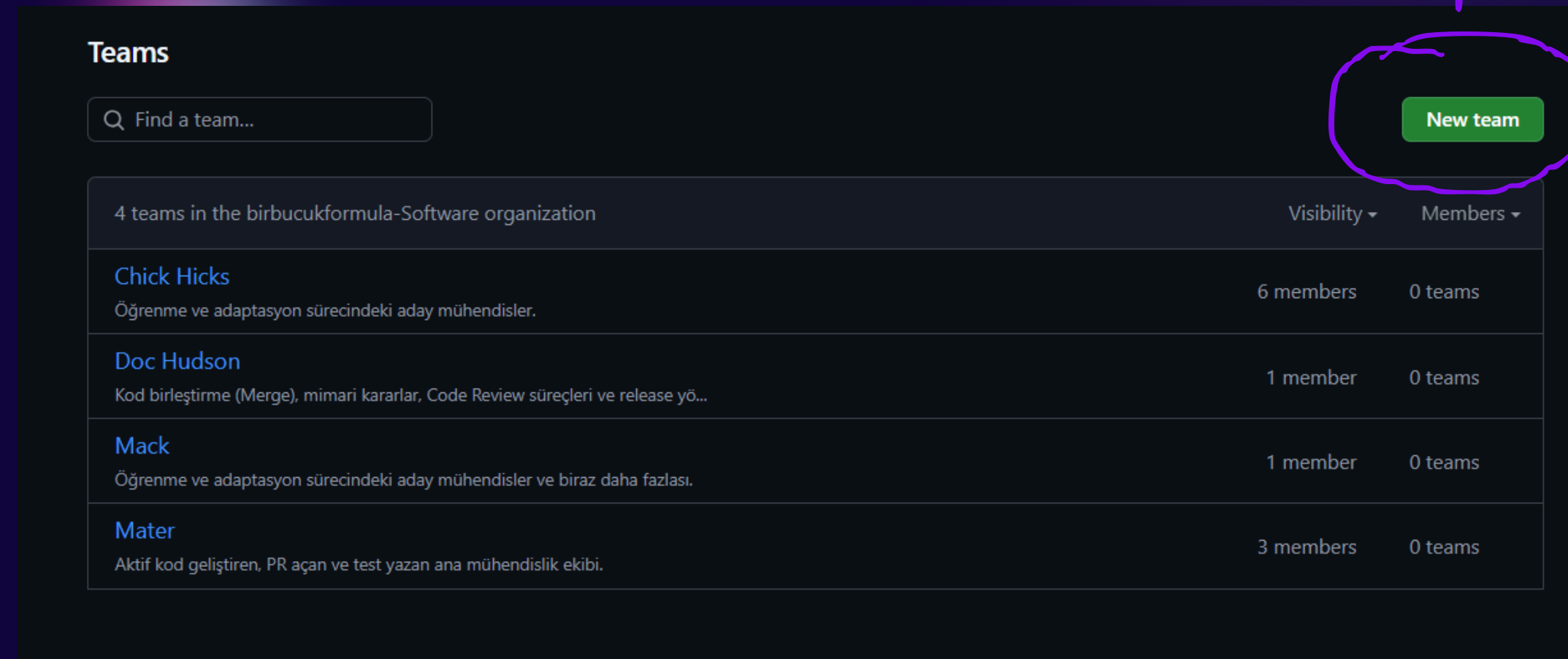
TAKIM (TEAM) MIMARISI

- Organizasyon içinde "Takımlar" kurmak, yetki yönetimini otomatize eder. Takımlar (Teams) sayesinde yetkileri toplu yönetiriz.
- Örneğin, yazılım ekibi ve elektrik ekibi diye 2 takım olsun. Yazılım ekibine ana koda tam erişim yetkisi verilirken, elektrik ekibine sadece dokümantasyon veya donanım şemalarının olduğu repolara erişim yetkisi verilir

NEDEN?

Çünkü herkesin her şeyi silme veya değiştirme yetkisi olması kaosa davetiyedir.

yeni takım kurmak için



ÜYE YÖNETİMİ

Takımdan çıkarmak: Kişi hala organizasyondadır ama o ekibin özel yetkilerini kaybeder.

Organizasyondan atmak: Kişinin şirket/takım ile tüm bağı kesilir, hiçbir özel depoyu göremez.

Find a member...

6 people in the birbucukformula-Software organization

Membership

	abdulmohammed4338-prog	Private	Member	1 team
	KruesnaPy KruesnaGod	Private	Member	1 team
	NecdetOzdemir	Public	Owner	4 teams
	Murat Can Ventux78	Private	Member	2 teams
	yyaseminkuru-cpu	Private	Member	1 team
	Zuleyhanurgunes	Private	Member	2 teams

Membership

- ✓ Everyone
- Owners
- Members

Private Member 1 team

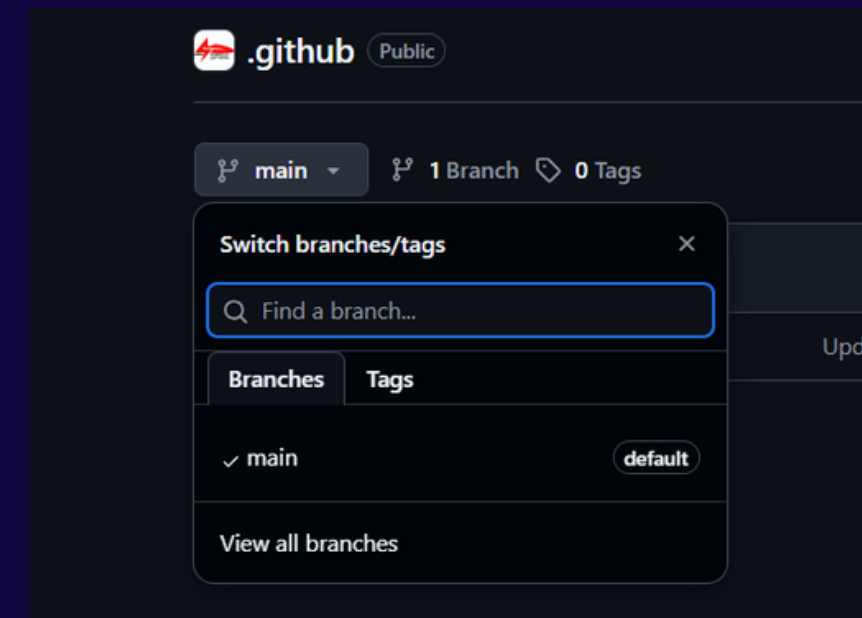
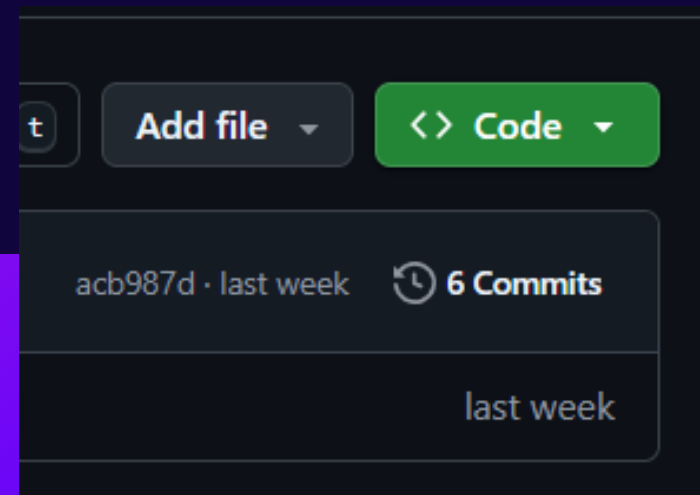
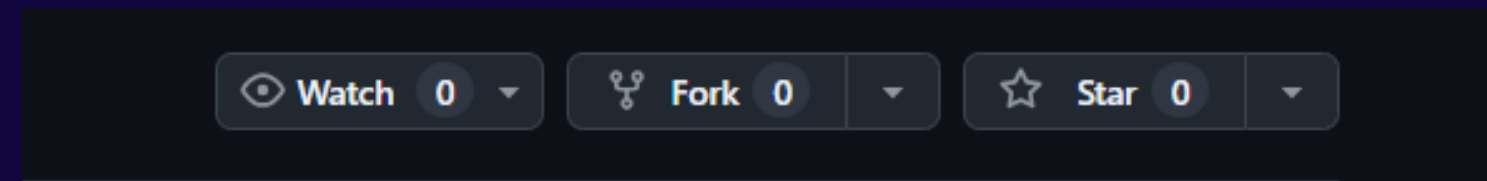
GITHUB TEMEL KAVRAMLAR

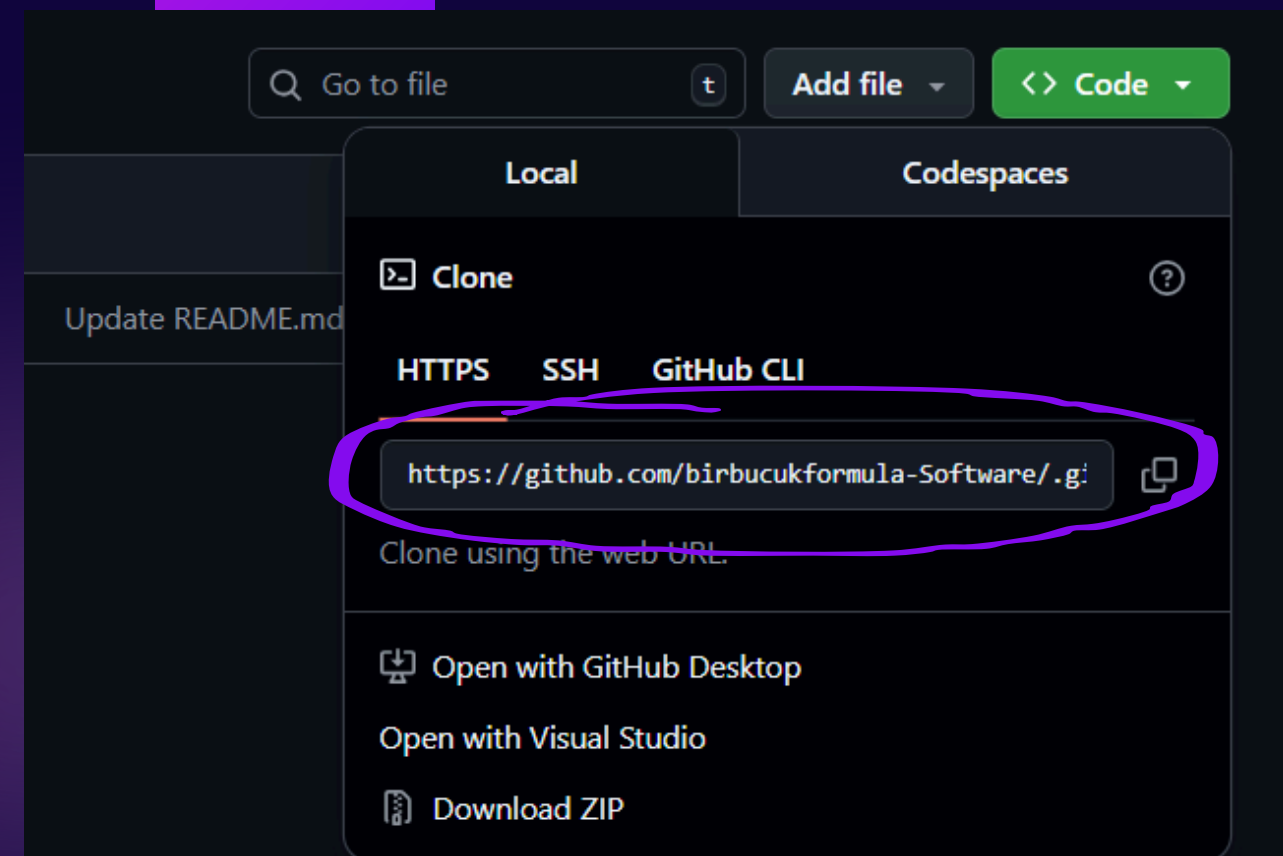
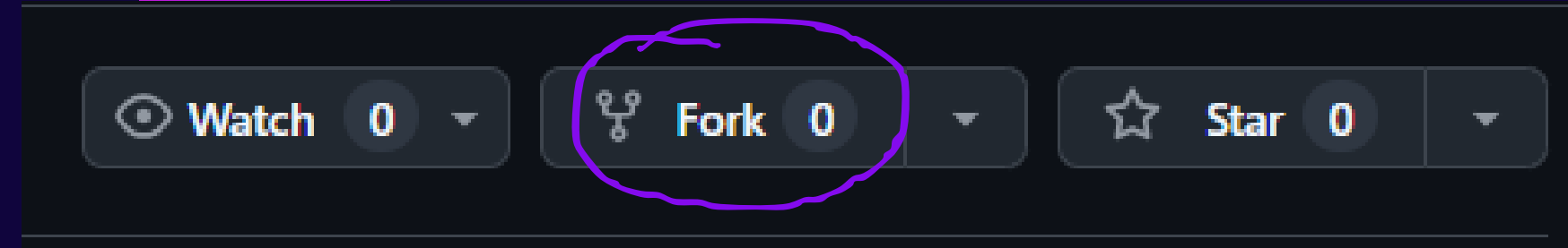
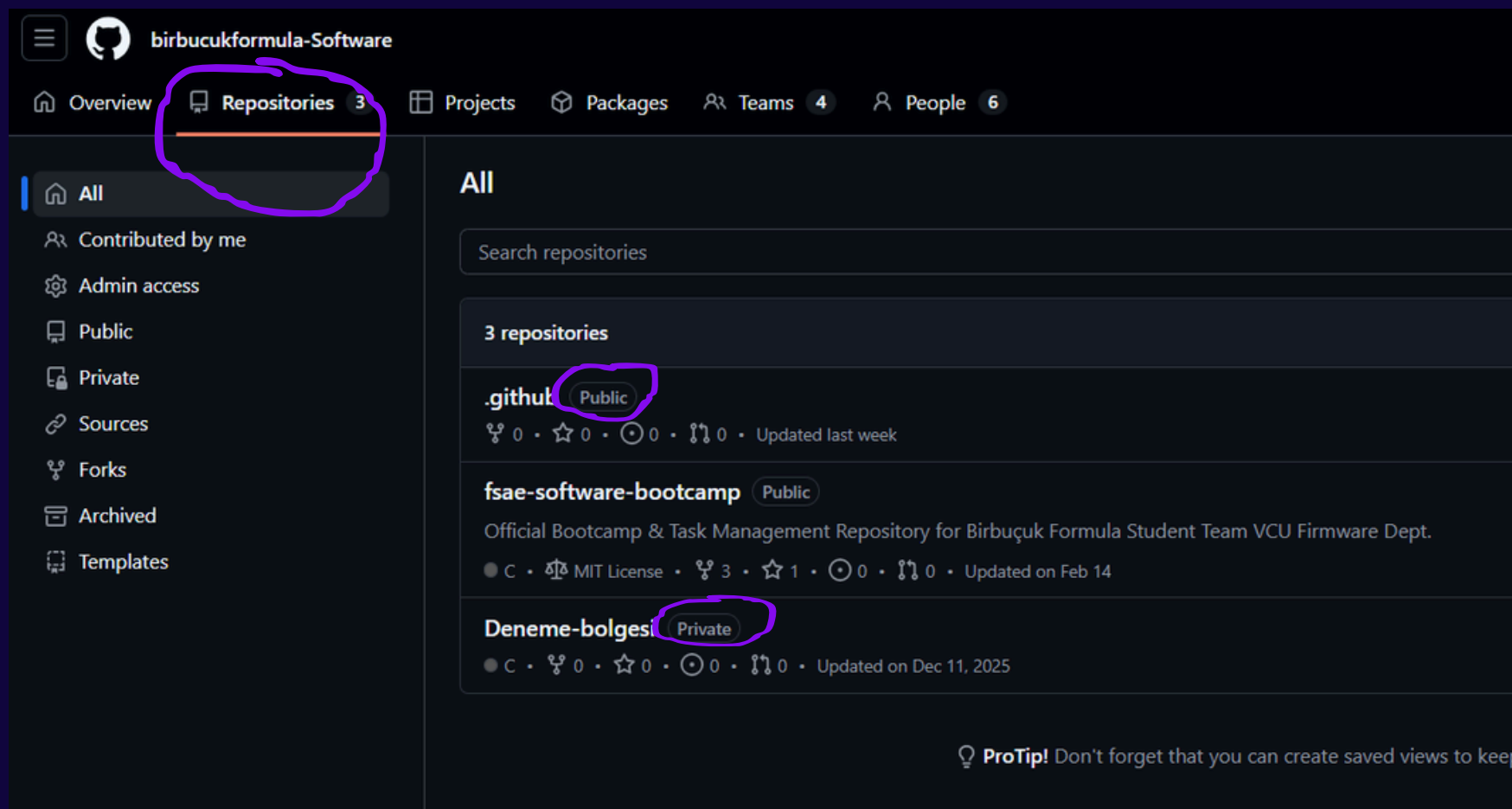
1. Repository (Repo)

2. Fork vs Clone

3. Branch (Dal)

4. Commit





REPO (REPOSITORY)

- Projenin dijital kutusudur. İçinde kodlar, dokümanlar ve tüm geçmiş (tarihçe) saklanır.
Public: herkes görebilir
Private: sadece davet edilenler görebilir.

FORK VS CLONE

- Fork:** Orijinal projenin bir kopyasını kendi GitHub hesabına almaktır (Sunucu taraflı).

Neden Önce Fork? Çünkü ana projeyi (upstream) bozma ihtimalini sıfıra indiririz. Önce kendi kopyamızda (fork) denemeler yaparız.

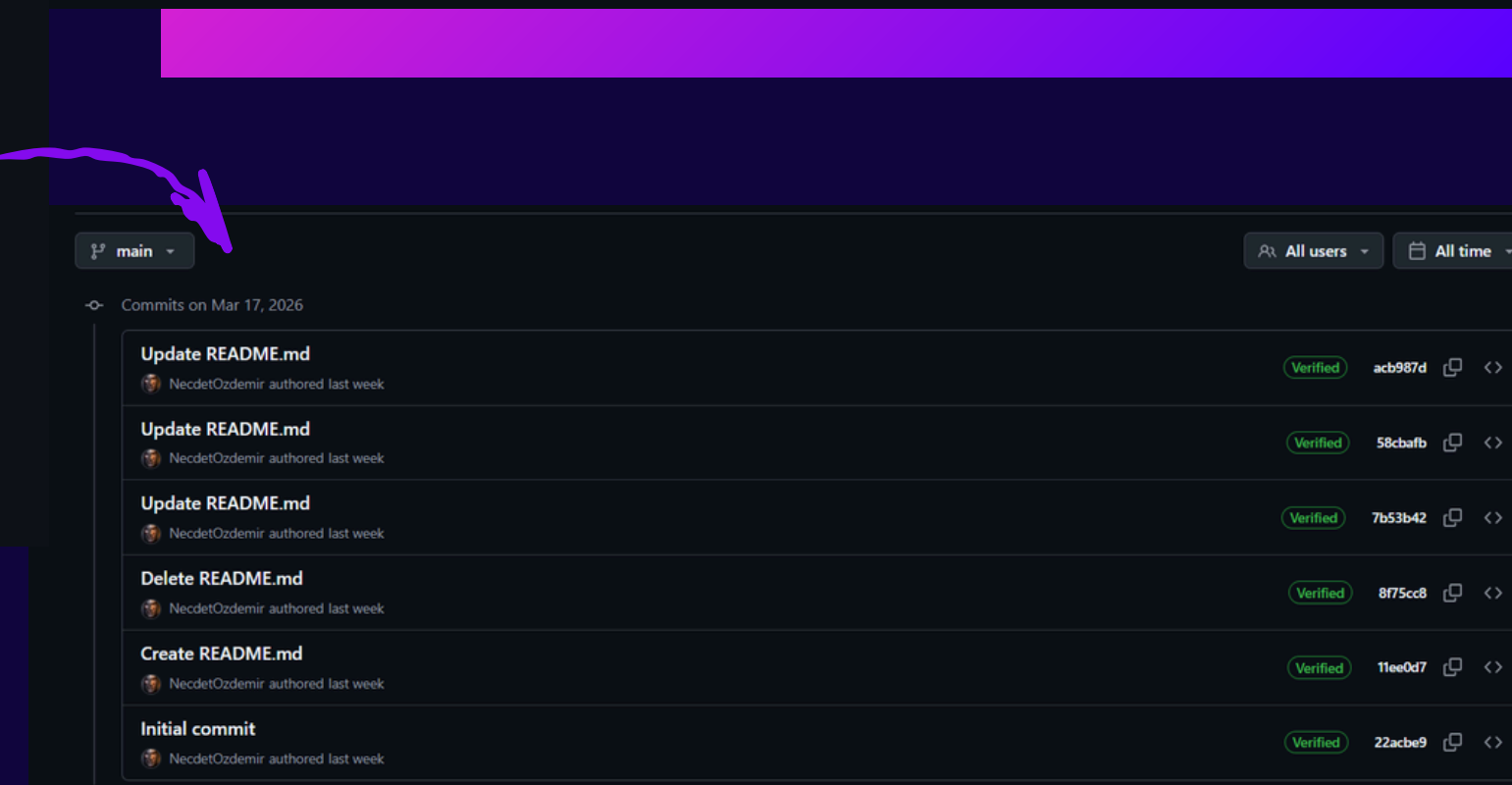
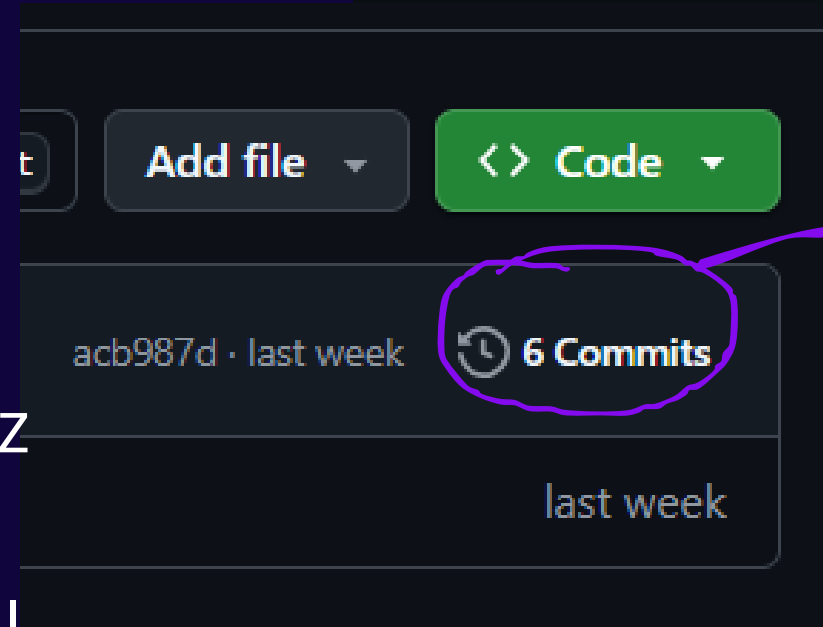
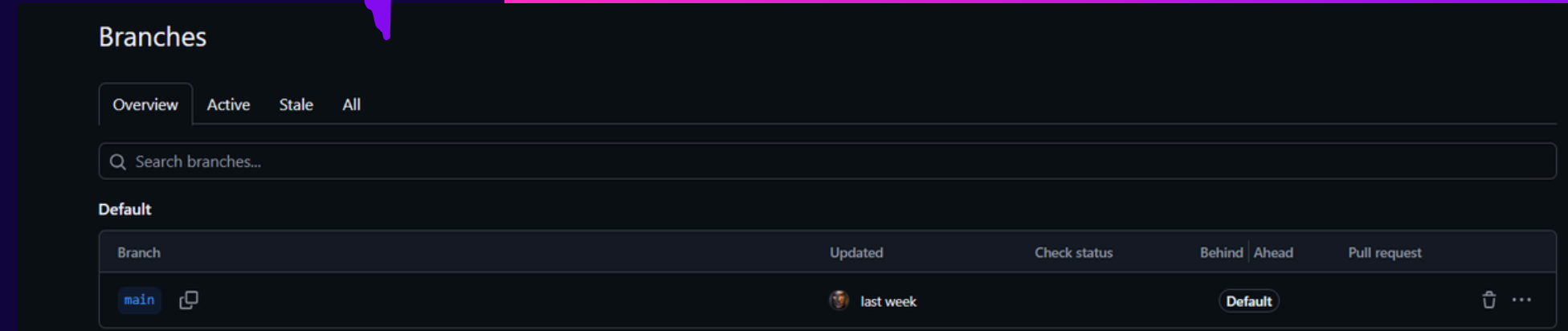
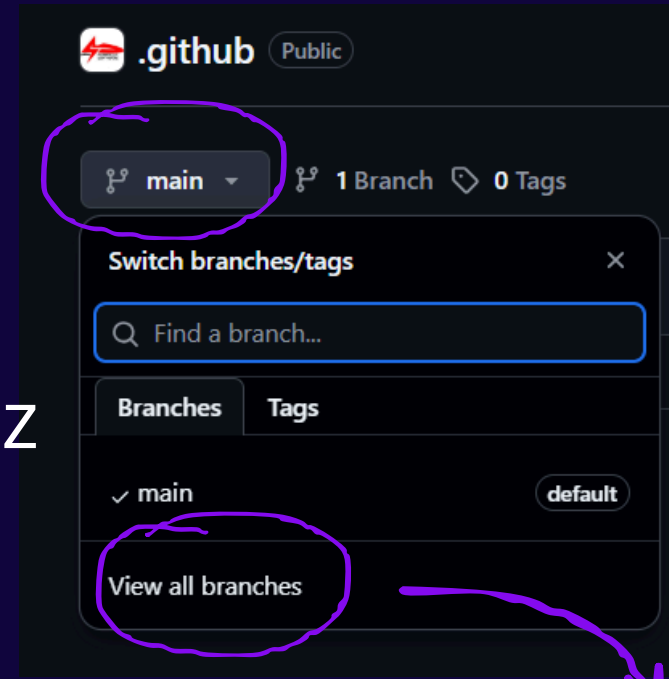
Clone: GitHub'daki bir depoyu kendi bilgisayarına indirmektir (Yerel taraflı).

BRANCH

- Ana dal (main) her zaman çalışan, hatasız sürümdür. Yeni bir özellik eklerken yan yola (yeni bir branch) gireriz.

Neden main yasak?

Çünkü bir hata yaparsan tüm projenin "kalbi" durur.



COMMIT

- Yapılan değişikliklerin o anki halini kaydetmektir.

Mühürlemek: Değişikliği geri dönülemez şekilde geçmişe kaydetmektir.

Mesaj Kuralı: "Güncelleme yaptım" değil, *feat: Motor kontrol algoritması eklendi* gibi açıklayıcı olmalıdır.

OPERASYONEL DÖNGÜ

1. **Fork Etme:** Bir repoyu kendi hesabına kopyalama.
2. **Clone:** Repoyu bilgisayara indirme.
3. **Work:** Dosya oluşturma ve değişiklik yapma.
4. **Push:** Kodları kendi Fork'una gönderme.
5. **Pull Request (PR):** Değişiklikleri ana repoya gönderme isteği.
6. **Sync Fork / Pull:** Ana repoda başkası değişiklik yaptığında, kendi bilgisayarımızı (git pull) ve Fork'umuzu nasıl güncelleriz?

FORK ETME

Orijinal projenin bir kopyasını kendi hesabımıza almaktır. Böylece ana projeye zarar vermeden çalışabiliriz.



Create a new fork

A *fork* is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.

Required fields are marked with an asterisk (*).

Owner *

 yyaseminkuru-cpu

Repository name *

/ .github

✓ .github is available.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description

0 / 350 characters

☒ Copy the `main` branch only

Contribute back to birbucukformula-Software/.github by adding your own branch. [Learn more.](#)

 You are creating a fork in your personal account.

Create fork

NASIL YAPILIR?

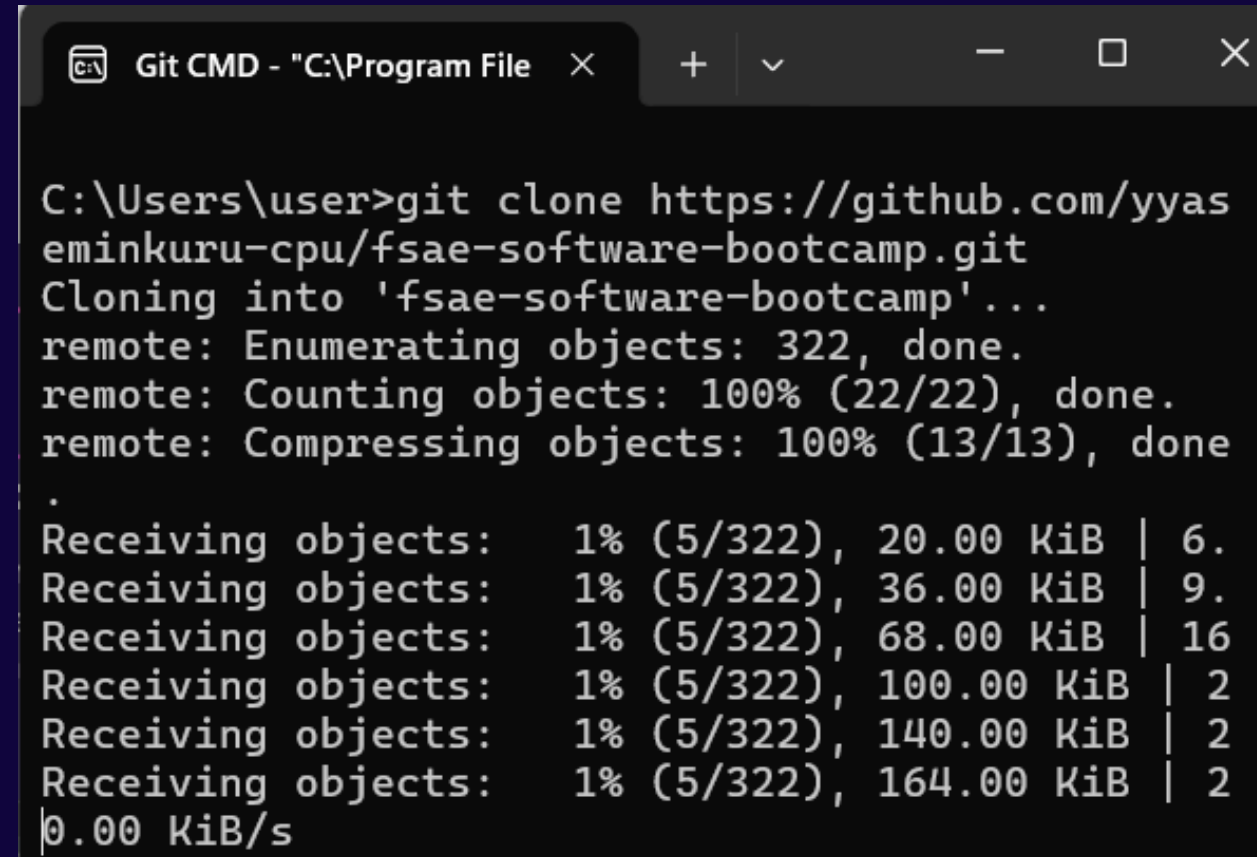
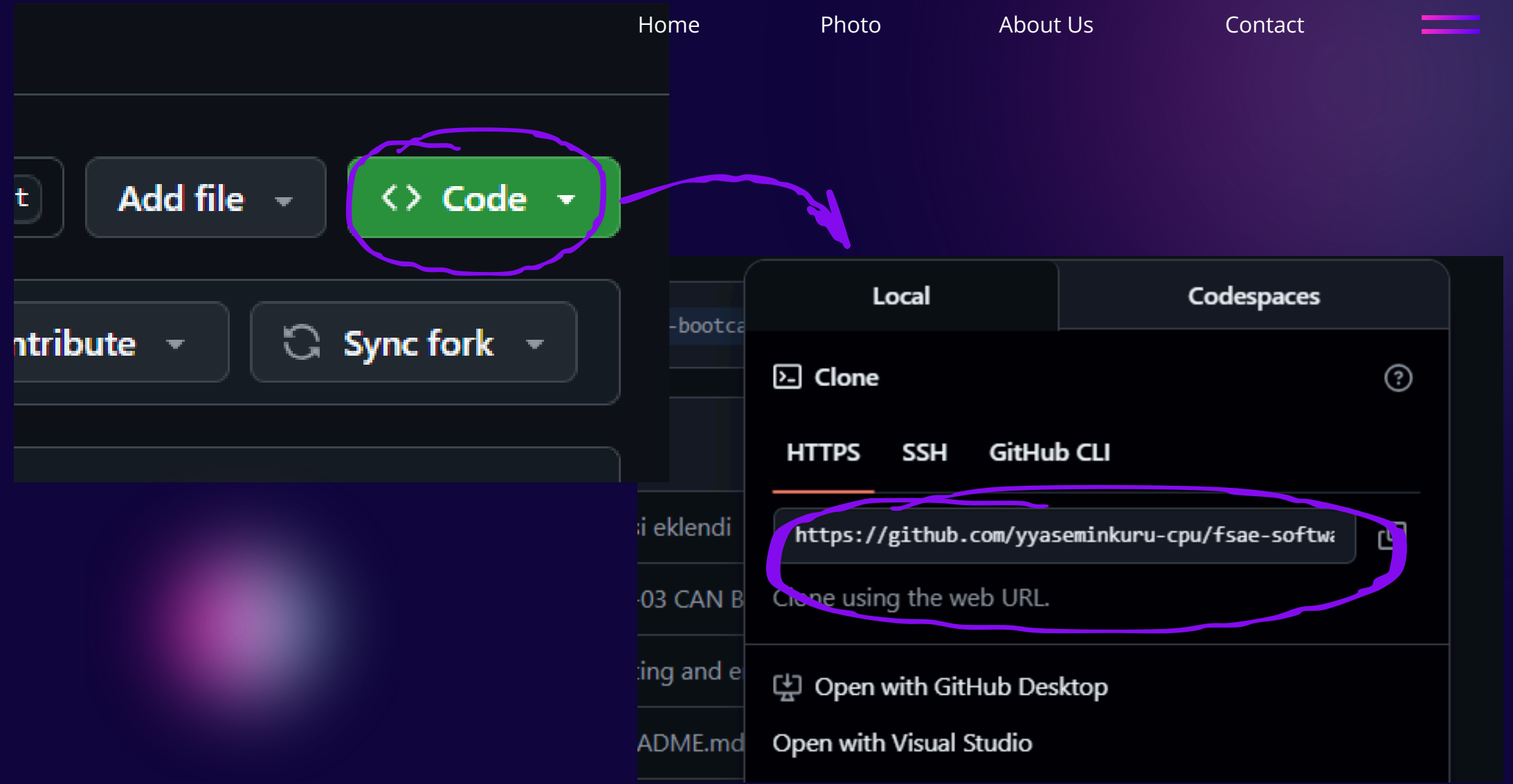
1. GitHub'da projenin sayfasına git.
2. Sağ üstte "**Fork**" butonuna bas.
3. Artık senin profilinde *kullanici-adin/repo-adi* şeklinde bir kopya var.

CLONE

- Kendi hesabındaki projeyi bilgisayarına indirmektir. Bu işlem projenin tüm geçmişini ve dosyalarını bilgisayarımıza kopyalar.

NASIL YAPILIR?

1. Fork ettiğin sayfada yeşil "**Code**" butonuna bas, oradaki HTTPS linkini kopyala.
2. Bilgisayarında terminali (veya CMD) aç ve şunu yaz: *git clone https://github.com/kullanici-adin/repo-adi.git*

A screenshot of a Git CMD terminal window. The command 'git clone https://github.com/yyaseminkuru-cpu/fsae-software-bootcamp.git' has been executed. The output shows the cloning process: 'Cloning into 'fsae-software-bootcamp'...', 'remote: Enumerating objects: 322, done.', 'remote: Counting objects: 100% (22/22), done.', 'remote: Compressing objects: 100% (13/13), done.', and a progress bar for receiving objects. The progress bar shows 1% (5/322) for the first five objects, with sizes ranging from 20.00 KiB to 164.00 KiB.

WORK

Main dalını bozmadan yeni bir özellik eklemektir. Asla main dalında çalışmıyoruz. Her yeni görev için yeni bir dal (branch) açıyoruz

NASIL YAPILIR?

1. Klasörün içine gir (*cd repo-adi*).
2. Yeni bir dal oluştur: *git checkout -b yeni-ozellik*.
3. Dosyalarını düzenle, kodunu yaz ve kaydet.

```
PS C:\Users\user\OneDrive\Masaüstü\fsae-software-bootcamp> git checkout -b feature/deneme
Switched to a new branch 'feature/deneme'
PS C:\Users\user\OneDrive\Masaüstü\fsae-software-bootcamp> |
```

PUSH

- Yaptığın değişiklikleri GitHub'daki kendi Fork'una göndermektir. Yaptığımız işi mühürleyip (commit), buluttaki depomuza (origin) geri gönderiyoruz.

 feature/deneme had recent pushes 19 seconds ago

Compare & pull request

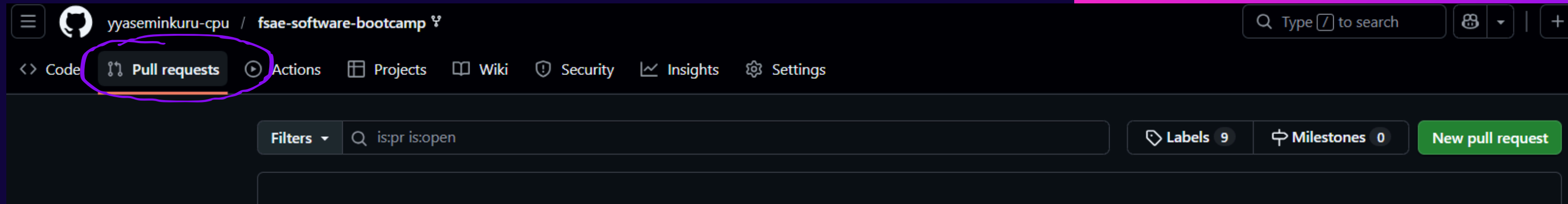
```
PS C:\Users\user\OneDrive\Masaüstü\fsae-software-bootcamp> git push origin feature/deneme
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 220 bytes | 110.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/deneme' on GitHub by visiting:
remote:   https://github.com/yyaseminkuru-cpu/fsae-software-bootcamp/pull/new/feature/deneme
remote:
To https://github.com/yyaseminkuru-cpu/fsae-software-bootcamp.git
 * [new branch]      feature/deneme -> feature/deneme
PS C:\Users\user\OneDrive\Masaüstü\fsae-software-bootcamp> |
```

NASIL YAPILIR?

- 1. *git add .* (Tüm değişiklikleri hazırla)
- 2. *git commit -m "feat: Yeni özellik eklendi"* (Mühürle)
- 3. *git push origin yeni-ozellik* (Gönder)

PULL REQUEST (PR)

- PR süreci, kod kalitesinin korunduğu yerdir. Takım lideri kodu inceler, hata varsa düzeltmeni ister, her şey tamamsa projeye dahil eder.



NASIL YAPILIR?

- 1. Kendi GitHub sayfana gir
- 2. Üstte sarı bir kutu çıkacak: **"Compare & pull request"**.
- 3. Ona bas. Değişikliklerini açıkla ve **"Create pull request"** de.

SYNC FORK / PULL

Biz çalışırken başkaları ana projeye bir şey eklediyse, o yenilikleri kendi bilgisayarına çekmektir. Çalışmaya başlamadan önce ana depodaki güncellemeleri kendi bilgisayarımıza çekmeliyiz

NASIL YAPILIR?

1. Önce ana depoyu (upstream) tanıtmak gerekir (genelde bir kez yapılır): *git remote add upstream https://github.com/ana-organizasyon/repo-adi.git*
2. Sonra güncellemeleri çek: *git pull upstream main*

```
PS C:\Users\user\OneDrive\Masaüstü\fsae-software-bootcamp> git push origin feature/deneme
Everything up-to-date
PS C:\Users\user\OneDrive\Masaüstü\fsae-software-bootcamp> |
```

CONFLICT (ÇAKIŞMA)

- Eğer sen bir dosyanın 10. satırını "A" yaparsan ve arkadaşın aynı anda aynı satırı "B" yapıp senden önce sisteme yüklerse, GitHub hangisinin doğru olduğunu bilemez.
- **Peki sonra ne olur?**
 - Git Durur: "Ben karar veremem, bir insan müdahale etmeli" diyerek işlemi kilitler.
 - İşaretleme: Dosyanın içine <<<<<< ve >>>>>> işaretlerini koyarak sana her iki seçeneği de gösterir.
 - Mühendis Kararı: Sen dosyayı açarsın; ya A'yı silersin, ya B'yi silersin ya da ikisini birleştirip yeni bir "C" yazarsın.
 - Çözüm: Gereksiz işaretleri sildikten sonra dosyayı kaydedip tekrar Commit ettiğinde kriz çözülmüş olur.

```
PS C:\Users\user\OneDrive\Masaüstü\fsae-software-  
-bootcamp\deneme.txt >  
<<<<<< HEAD  
Satır 10: [yyaseminkuru-cpu] - Bu benim  
değişikliğim, A seçeneği (Mevcut).  
=====  
Satır 10: [Diğer Kullanıcı] - Bu arkadaşımın  
değişikliği, B seçeneği (Gelen).  
>>>>>> main  
  
[HATA! Git Durdu ve Kilitlendi]  
Dosyanın içine <<<<<< ve >>>>>> işaretlerini  
koydu. Müdahale etmenizi bekliyor.  
PS C:\Users\user\...\fsae-software-bootcamp > |
```


TEŞEKKÜRLER